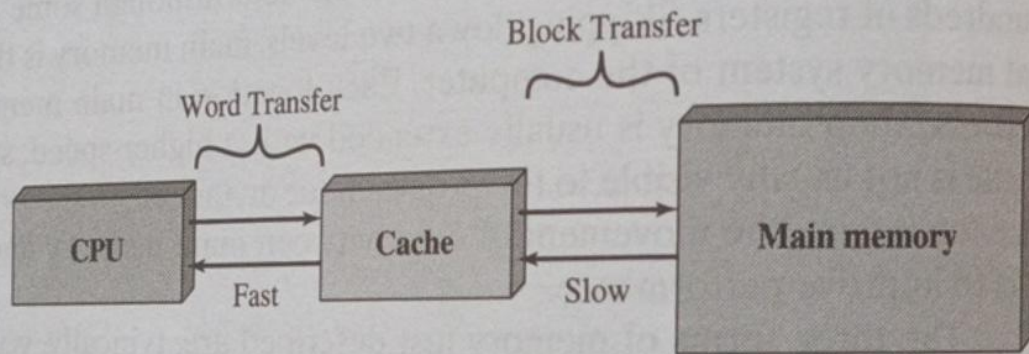


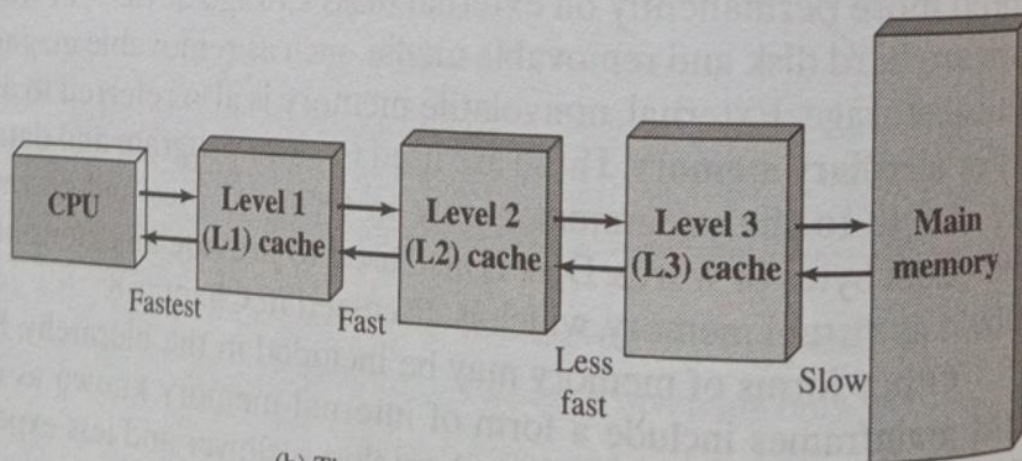
ware can
Appendix 4A examines the pos-
structures.

4.2 CACHE MEMORY PRINCIPLES

Cache memory is intended to give memory speed approaching that of the fastest memories available, and at the same time provide a large memory size at the price of less expensive types of semiconductor memories. The concept is illustrated in Figure 4.3a. There is a relatively large and slow main memory together with a smaller, faster cache memory. The cache contains a copy of portions of main memory. When the processor attempts to read a word of memory, a check is made



(a) Single cache



(b) Three-level cache organization

Figure 4.3 Cache and Main Memory

²Disk cache is generally a purely software technique and is not examined in this book. See [STALON] for a discussion.

determine if the word is in the cache. If so, the word is delivered to the processor. If not, a block of main memory, consisting of some fixed number of words, is read into the cache and then the word is delivered to the processor. Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that there will be future references to that same memory location or to other words in the block.

Figure 4.3b depicts the use of multiple levels of cache. The L2 cache is slower and typically larger than the L1 cache, and the L3 cache is slower and typically larger than the L2 cache.

Figure 4.4 depicts the structure of a cache/main-memory system. Main memory consists of up to 2^n addressable words, with each word having a unique n -bit address. For mapping purposes, this memory is considered to consist of a number of fixed-length blocks of K words each. That is, there are $M = 2^n/K$ blocks in main memory. The cache consists of m blocks, called lines.³ Each line contains K words, plus a tag of a few bits. Each line also includes control bits (not shown), such as a bit to indicate

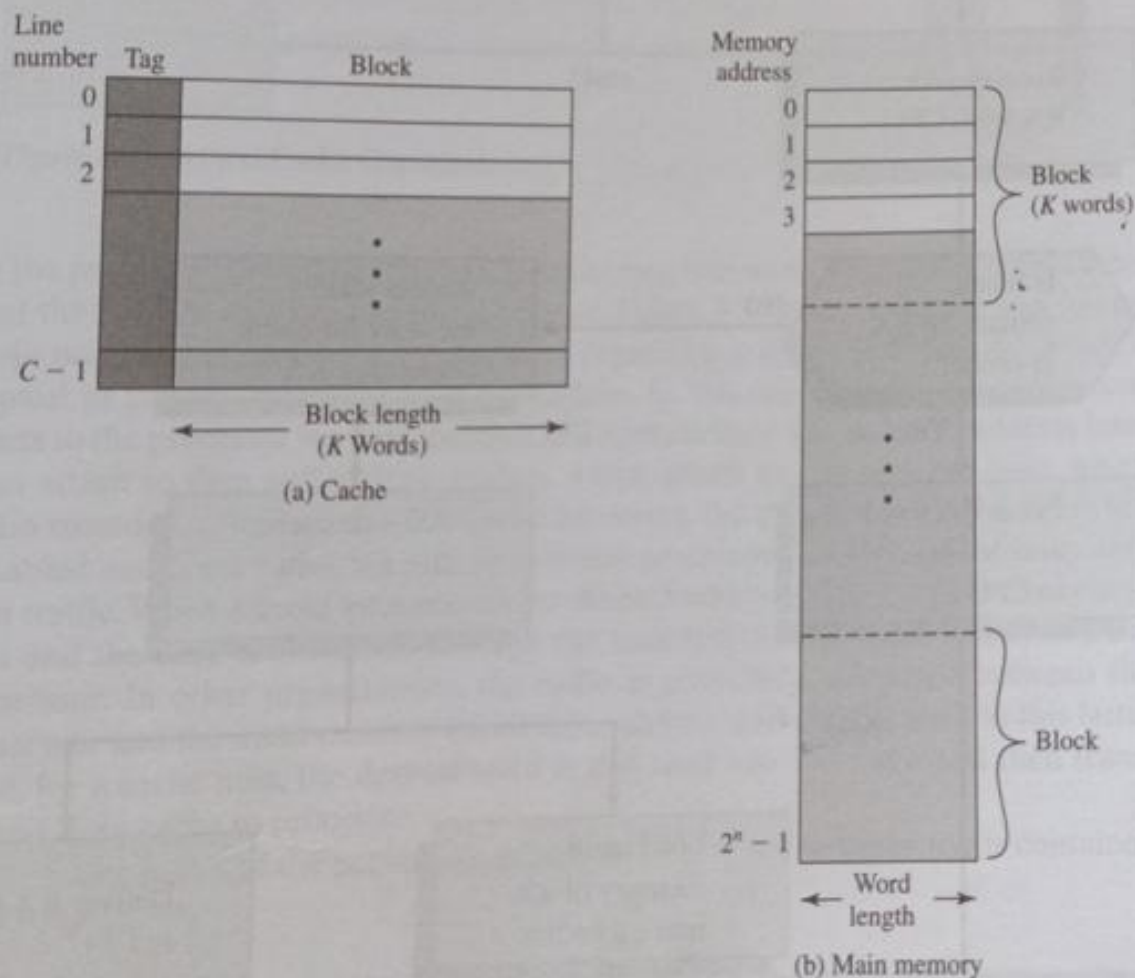


Figure 4.4 Cache/Main Memory Structure

³In referring to the basic unit of the cache, the term *line* is used, rather than the term *block*, for two reasons: (1) to avoid confusion with a main memory block, which contains the same number of data words as a cache line; and (2) because a cache line includes not only K words of data, just as a main memory block, but also include tag and control bits.

whether the line has been modified since being loaded into the cache. The length of a line, not including tag and control bits, is the **line size**. The line size may be as small as 32 bits, with each "word" being a single byte; in this case the line size is 4 bytes. The number of lines is considerably less than the number of main memory blocks ($m \ll M$). At any time, some subset of the blocks of memory resides in lines in the cache. If a word in a block of memory is read, that block is transferred to one of the lines of the cache. Because there are more blocks than lines, an individual line cannot be uniquely and permanently dedicated to a particular block. Thus, each line includes a **tag** that identifies which particular block is currently being stored. The tag is usually a portion of the main memory address, as described later in this section.

Figure 4.5 illustrates the read operation. The processor generates the read address (RA) of a word to be read. If the word is contained in the cache, it is delivered

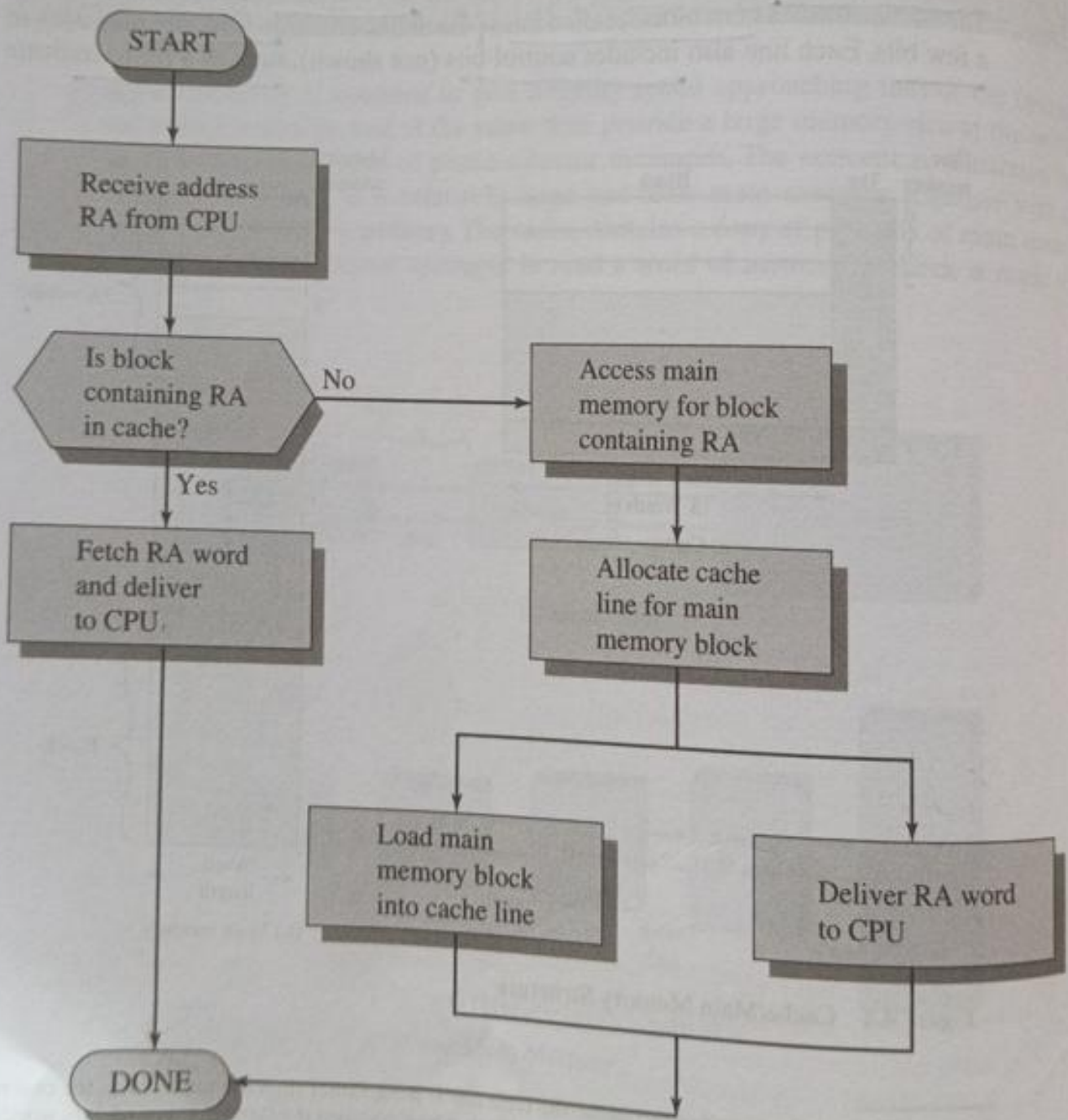


Figure 4.5 Cache Read Operation

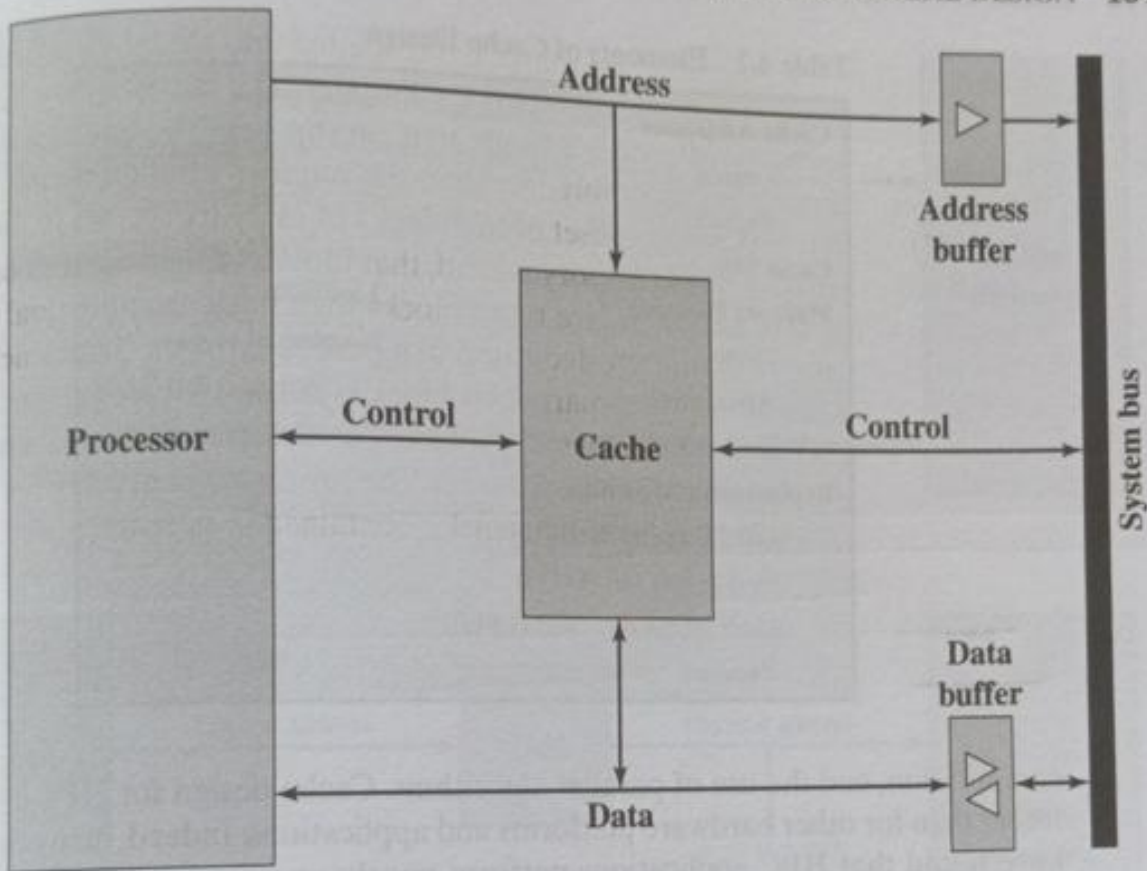


Figure 4.6 Typical Cache Organization

to the processor. Otherwise, the block containing that word is loaded into the cache, and the word is delivered to the processor. Figure 4.5 shows these last two operations occurring in parallel and reflects the organization shown in Figure 4.6, which is typical of contemporary cache organizations. In this organization, the cache connects to the processor via data, control, and address lines. The data and address lines also attach to data and address buffers, which attach to a system bus from which main memory is reached. When a cache hit occurs, the data and address buffers are disabled and communication is only between processor and cache, with no system bus traffic. When a cache miss occurs, the desired address is loaded onto the system bus and the data are returned through the data buffer to both the cache and the processor. In other organizations, the cache is physically interposed between the processor and the main memory for all data, address, and control lines. In this latter case, for a cache miss, the desired word is first read into the cache and then transferred from cache to processor.

A discussion of the performance parameters related to cache use is contained in Appendix 4A.

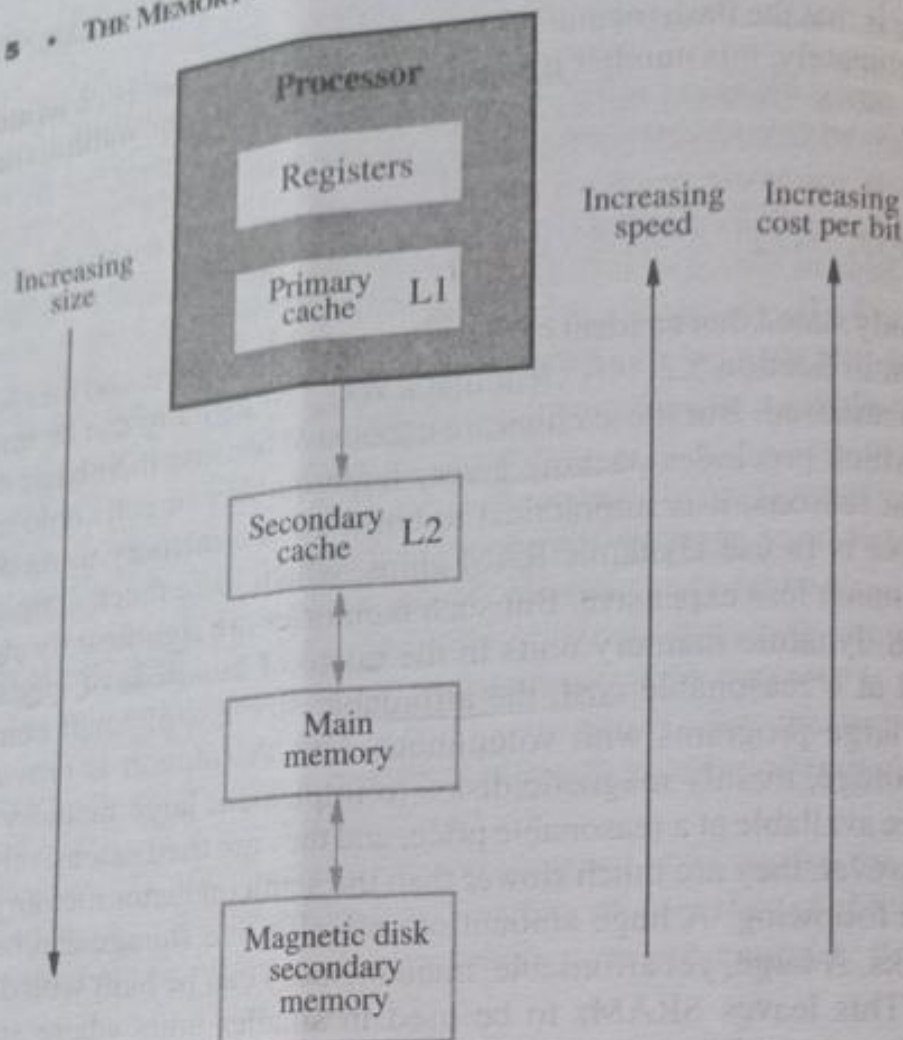


Figure 5.13 Memory hierarchy.

DIMMs, or RIMMs. The main memory is much larger but significantly slower than the cache memory. In a typical computer, the access time for the main memory is about ten times longer than the access time for the L1 cache.

Disk devices provide a huge amount of inexpensive storage. They are very slow compared to the semiconductor devices used to implement the main memory. We will discuss disk technology in Section 5.9.

During program execution, the speed of memory access is of utmost importance. The key to managing the operation of the hierarchical memory system in Figure 5.13 is to bring the instructions and data that will be used in the near future as close to the processor as possible. This can be done by using the mechanisms presented in the sections that follow. We begin with a detailed discussion of cache memories.

5.5 CACHE MEMORIES

The speed of the main memory is very low in comparison with the speed of modern processors. For good performance, the processor cannot spend much of its time waiting to access instructions and data in main memory. Hence, it is important to devise a scheme

that reduces the time needed to access the necessary information. Since the speed of the main memory unit is limited by electronic and packaging constraints, the solution must be sought in a different architectural arrangement. An efficient solution is to use a fast cache memory which essentially makes the main memory appear to the processor to be faster than it really is.

The effectiveness of the cache mechanism is based on a property of computer programs called locality of reference. Analysis of programs shows that most of their execution time is spent on routines in which many instructions are executed repeatedly. These instructions may constitute a simple loop, nested loops, or a few procedures that repeatedly call each other. The actual detailed pattern of instruction sequencing is not important — the point is that many instructions in localized areas of the program are executed repeatedly during some time period, and the remainder of the program is accessed relatively infrequently. This is referred to as locality of reference. It manifests itself in two ways: temporal and spatial. The first means that a recently executed instruction is likely to be executed again very soon. The spatial aspect means that instructions in close proximity to a recently executed instruction (with respect to the instructions' addresses) are also likely to be executed soon.

If the active segments of a program can be placed in a fast cache memory, then the total execution time can be reduced significantly. Conceptually, operation of a cache memory is very simple. The memory control circuitry is designed to take advantage of the property of locality of reference. The temporal aspect of the locality of reference suggests that whenever an information item (instruction or data) is first needed, this item should be brought into the cache where it will hopefully remain until it is needed again. The spatial aspect suggests that instead of fetching just one item from the main memory to the cache, it is useful to fetch several items that reside at adjacent addresses as well. We will use the term *block* to refer to a set of contiguous address locations of some size. Another term that is often used to refer to a cache block is *cache line*.

Consider the simple arrangement in Figure 5.14. When a Read request is received from the processor, the contents of a block of memory words containing the location specified are transferred into the cache one word at a time. Subsequently, when the program references any of the locations in this block, the desired contents are read directly from the cache. Usually, the cache memory can store a reasonable number of blocks at any given time, but this number is small compared to the total number of

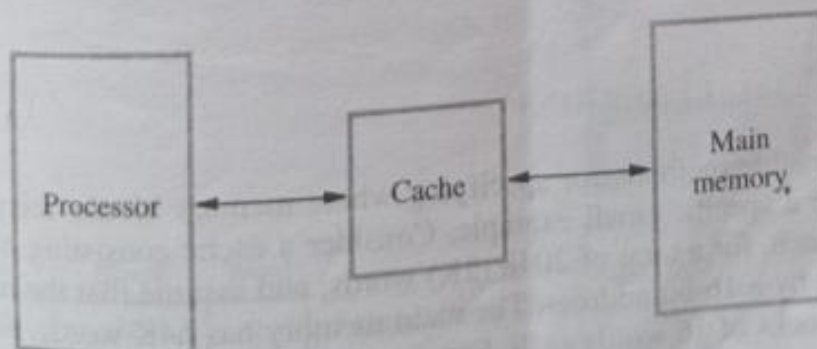


Figure 5.14 Use of a cache memory.

cache
↓
locality
of Reference
↙ ↘
temporal spatial

blocks in the main memory. The correspondence between the main memory blocks and those in the cache is specified by a mapping function. When the cache is full and a memory word (instruction or data) that is not in the cache is referenced, the cache control hardware must decide which block should be removed to create space for the new block that contains the referenced word. The collection of rules for making this decision constitutes the *replacement algorithm*.

The processor does not need to know explicitly about the existence of the cache. It simply issues Read and Write requests using addresses that refer to locations in the memory. The cache control circuitry determines whether the requested word currently exists in the cache. If it does, the Read or Write operation is performed on the appropriate cache location. In this case, a *read* or *write hit* is said to have occurred. In a Read operation, the main memory is not involved. For a Write operation, the system can proceed in two ways. In the first technique, called the *write-through* protocol, the cache location and the main memory location are updated simultaneously. The second technique is to update only the cache location and to mark it as updated with an associated flag bit, often called the *dirty* or *modified* bit. The main memory location of the word is updated later, when the block containing this marked word is to be removed from the cache to make room for a new block. This technique is known as the *write-back*, or *copy-back*, protocol. The write-through protocol is simpler, but it results in unnecessary Write operations in the main memory when a given cache word is updated several times during its cache residency. Note that the write-back protocol may also result in unnecessary Write operations because when a cache block is written back to the memory all words of the block are written back, even if only a single word has been changed while the block was in the cache.

When the addressed word in a Read operation is not in the cache, a *read miss* occurs. The block of words that contains the requested word is copied from the main memory into the cache. After the entire block is loaded into the cache, the particular word requested is forwarded to the processor. Alternatively, this word may be sent to the processor as soon as it is read from the main memory. The latter approach, which is called *load-through*, or *early restart*, reduces the processor's waiting period somewhat, but at the expense of more complex circuitry.

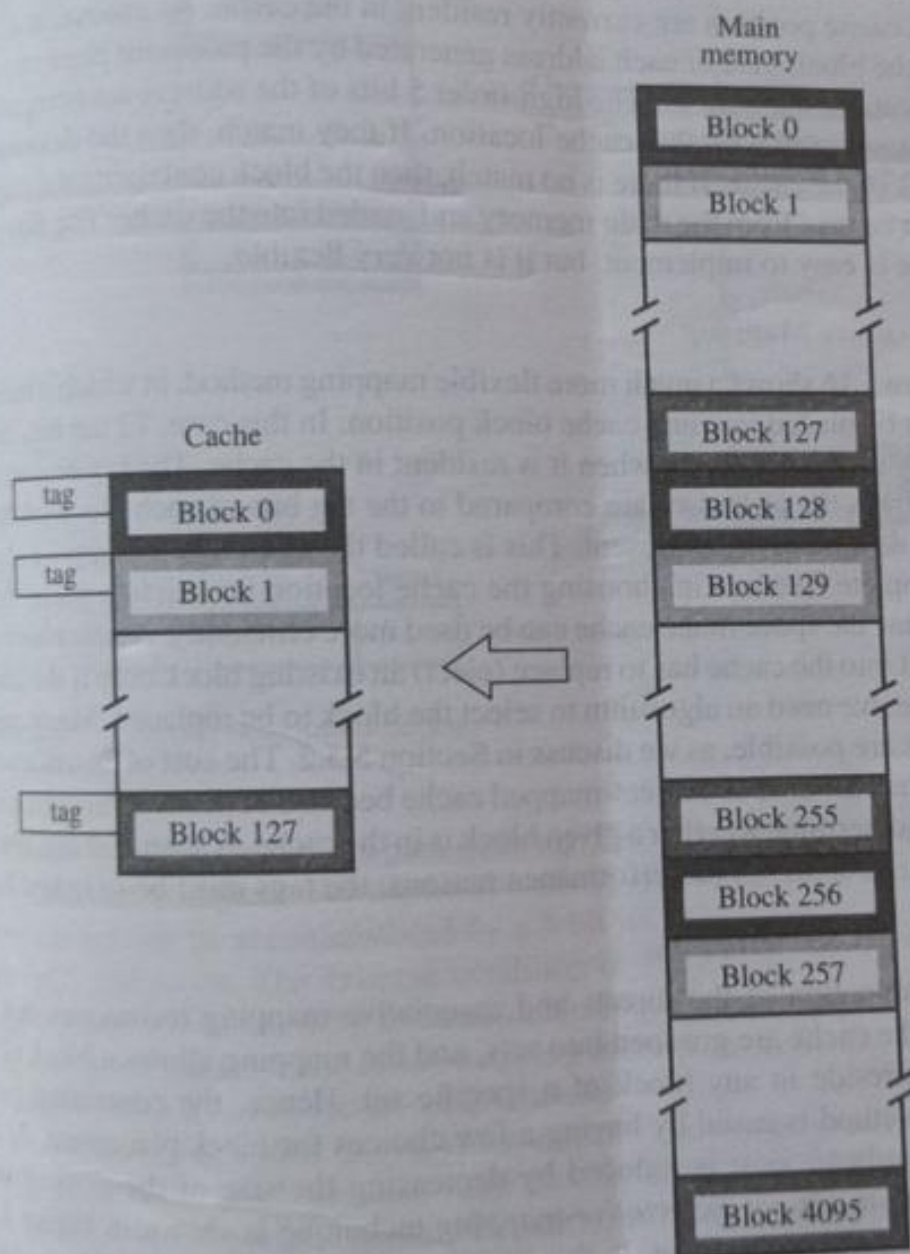
During a Write operation, if the addressed word is not in the cache, a *write miss* occurs. Then, if the write-through protocol is used, the information is written directly into the main memory. In the case of the write-back protocol, the block containing the addressed word is first brought into the cache, and then the desired word in the cache is overwritten with the new information.

5.5.1 MAPPING FUNCTIONS

To discuss possible methods for specifying where memory blocks are placed in the cache, we use a specific small example. Consider a cache consisting of 128 blocks of 16 words each, for a total of 2048 (2K) words, and assume that the main memory is addressable by a 16-bit address. The main memory has 64K words, which we will view as 4K blocks of 16 words each.

Direct Mapping

The simplest way to determine cache locations in which to store memory blocks is the *direct-mapping* technique. In this technique, block j of the main memory maps onto block j modulo 128 of the cache, as depicted in Figure 5.15. Thus, whenever one of the main memory blocks 0, 128, 256, ... is loaded in the cache, it is stored in cache block 0. Blocks 1, 129, 257, ... are stored in cache block 1, and so on. Since more than one memory block is mapped onto a given cache block position, contention may arise for that position even when the cache is not full. For example, instructions of a



Tag	Block	Word	Main memory address
5	7	4	

Figure 5.15 Direct-mapped cache.

program may start in block 1 and continue in block 129, possibly after a branch. A program is executed, both of these blocks must be transferred to the block-1 position in the cache. Contention is resolved by allowing the new block to overwrite the current resident block. In this case, the replacement algorithm is trivial.

Placement of a block in the cache is determined from the memory address. The memory address can be divided into three fields, as shown in Figure 5.15. The low-order 4 bits select one of 16 words in a block. When a new block enters the cache, the cache block field determines the cache position in which this block must be stored. The high-order 5 bits of the memory address of the block are stored in 5 tag bits associated with its location in the cache. They identify which of the 32 blocks that are mapped into this cache position are currently resident in the cache. As execution proceeds, the 7-bit cache block field of each address generated by the processor points to a particular block location in the cache. The high-order 5 bits of the address are compared with the tag bits associated with that cache location. If they match, then the desired word is in that block of the cache. If there is no match, then the block containing the required word must first be read from the main memory and loaded into the cache. The direct-mapping technique is easy to implement, but it is not very flexible.

Associative Mapping

Figure 5.16 shows a much more flexible mapping method, in which a main memory block can be placed into any cache block position. In this case, 12 tag bits are required to identify a memory block when it is resident in the cache. The tag bits of an address received from the processor are compared to the tag bits of each block of the cache to see if the desired block is present. This is called the *associative-mapping* technique. It gives complete freedom in choosing the cache location in which to place the memory block. Thus, the space in the cache can be used more efficiently. A new block that has to be brought into the cache has to replace (eject) an existing block only if the cache is full. In this case, we need an algorithm to select the block to be replaced. Many replacement algorithms are possible, as we discuss in Section 5.5.2. The cost of an associative cache is higher than the cost of a direct-mapped cache because of the need to search all 128 tag patterns to determine whether a given block is in the cache. A search of this kind is called an *associative search*. For performance reasons, the tags must be searched in parallel.

Set-Associative Mapping

A combination of the direct- and associative-mapping techniques can be used. Blocks of the cache are grouped into sets, and the mapping allows a block of the main memory to reside in any block of a specific set. Hence, the contention problem of the direct method is eased by having a few choices for block placement. At the same time, the hardware cost is reduced by decreasing the size of the associative search. An example of this *set-associative-mapping* technique is shown in Figure 5.17 for a cache with two blocks per set. In this case, memory blocks 0, 64, 128, ..., 4032 map into cache set 0, and they can occupy either of the two block positions within this set. Having 64 sets means that the 6-bit set field of the address determines which set of the cache might contain the desired block. The tag field of the address must then be compared associatively to the tags of the two blocks of the set to check if the desired block is present. This two-way associative search is simple to implement.

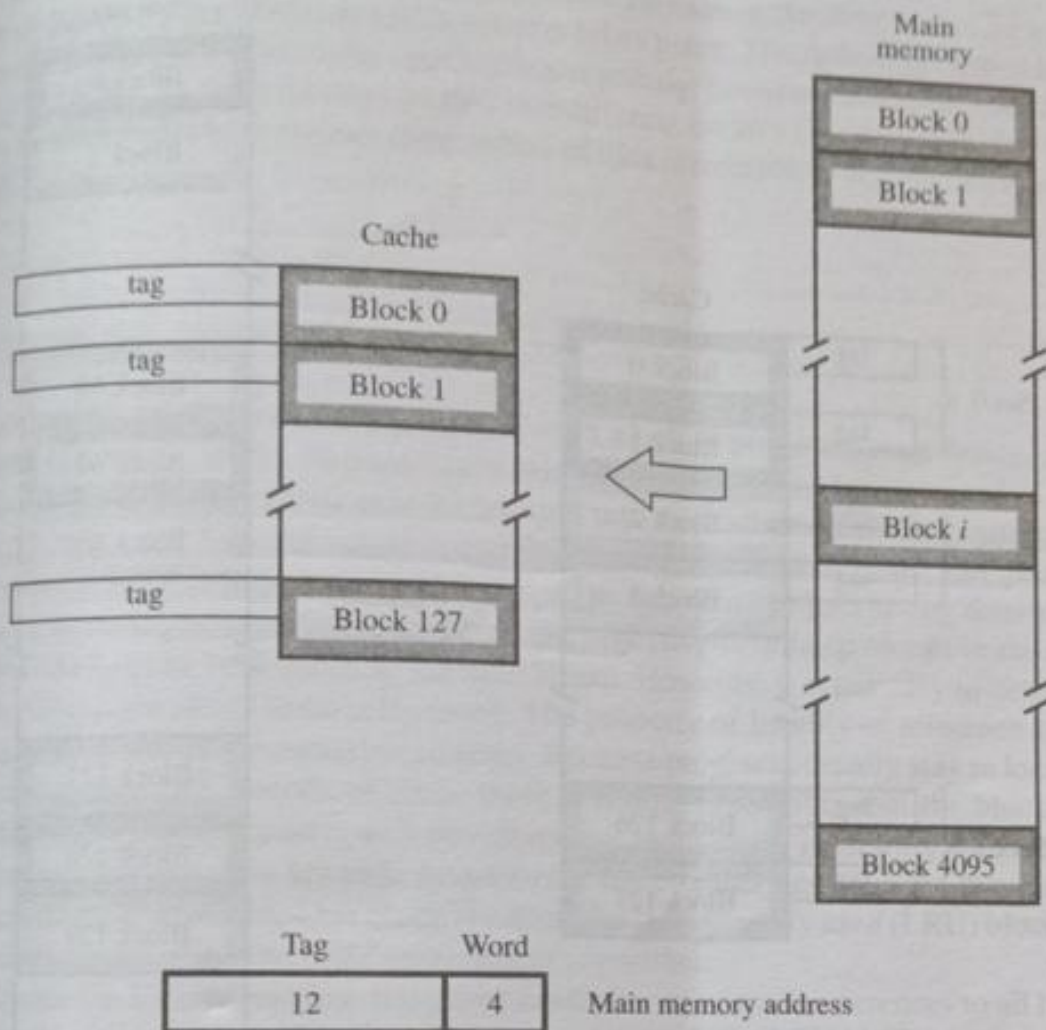
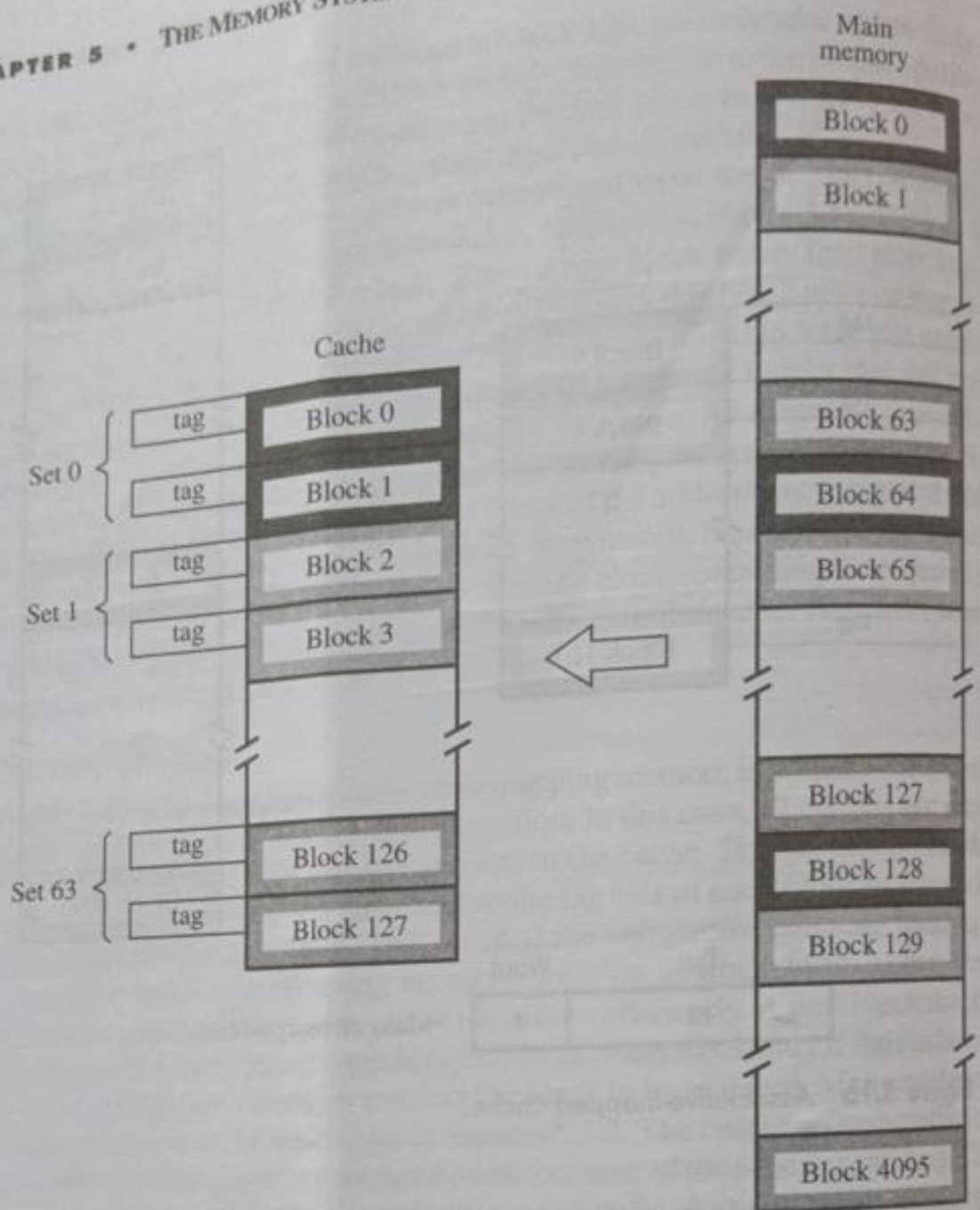


Figure 5.16 Associative-mapped cache.

The number of blocks per set is a parameter that can be selected to suit the requirements of a particular computer. For the main memory and cache sizes in Figure 5.17, four blocks per set can be accommodated by a 5-bit set field, eight blocks per set by a 4-bit set field, and so on. The extreme condition of 128 blocks per set requires no set bits and corresponds to the fully associative technique, with 12 tag bits. The other extreme of one block per set is the direct-mapping method. A cache that has k blocks per set is referred to as a k -way set-associative cache.

One more control bit, called the *valid bit*, must be provided for each block. This bit indicates whether the block contains valid data. It should not be confused with the modified, or dirty, bit mentioned earlier. The dirty bit, which indicates whether the block has been modified during its cache residency, is needed only in systems that do not use the write-through method. The valid bits are all set to 0 when power is initially applied to the system or when the main memory is loaded with new programs and data from the disk. Transfers from the disk to the main memory are carried out by a DMA mechanism. Normally, they bypass the cache for both cost and performance reasons. The valid bit of a particular cache block is set to 1 the first time this block is loaded.



Tag	Set	Word
6	6	4

Main memory address

Figure 5.17 Set-associative-mapped cache with two blocks per set.

from the main memory. Whenever a main memory block is updated by a source that bypasses the cache, a check is made to determine whether the block being loaded is currently in the cache. If it is, its valid bit is cleared to 0. This ensures that *stale data* will not exist in the cache.

A similar difficulty arises when a DMA transfer is made from the main memory to the disk, and the cache uses the write-back protocol. In this case, the data in the memory might not reflect the changes that may have been made in the cached copy.

One solution to this problem is to *flush* the cache by forcing the dirty data to be written back to the memory before the DMA transfer takes place. The operating system can do this easily, and it does not affect performance greatly, because such disk transfers do not occur often. This need to ensure that two different entities (the processor and DMA subsystems in this case) use the same copies of data is referred to as a *cache-coherence* problem.

Module 3

Mapping $\rightarrow$ Direct Mapping
 $\rightarrow$ Associate mapping
 $\rightarrow$ Set Associate Mapping

Address Mapping Technique

$\rightarrow$ decide which block of main memory has to be placed where in the cache memory

3 Types

- > Direct Mapping
- > Associative Mapping
- > Set Associative Mapping

1. Direct Mapping

k^{th} block of main mly is placed in $(k^{\text{th}} \text{ Mod } n^{\text{th}})$ block of cache memory

$(k^{\text{th}} \text{ Mod } n)$ $\leftarrow$ Main mly
 cache $k^{\text{th}} \text{ block}$

Eg:- Assume

32 words in Main memory
 16 words in cache memory.
 Block Size = 4 word

in 1 block contains 4 word

copying 4 words in cache memory at a time

Physical address = 5 bit

32 words = $2^5 = 32$
 $\rightarrow$ 5 bits

Physical address based on number of words in Main memory

cache memory

0
1
2
3
...

16 words

$$\text{block size} = 16/4 = 4.$$

$$\text{No of block (N)} = \underline{4}$$

16 words $\leftarrow$ Each block contain 4 words

$$\text{Number of block } N = \underline{4}$$

How to placed block in mly to cache mly

$$\text{block Number} = 0 \text{ (K)}$$

$$K^{\text{th}} \text{ mode } N =$$

$$0 \text{ mode } 4 = 0 //$$

0th block placed in 0th block in cache. 1th block placed in 1th block in cache.

0
1
2
3

0	w ₀ w ₁ w ₂ w ₃
1	w ₄ w ₅ w ₆ w ₇
2	w ₈ w ₉ w ₁₀ w ₁₁
3	w ₁₂ w ₁₃ w ₁₄ w ₁₅
4	w ₁₆ w ₁₇ w ₁₈ w ₁₉
5	w ₂₀ w ₂₁ w ₂₂ w ₂₃
6	w ₂₄ w ₂₅ w ₂₆ w ₂₇
7	w ₂₈ w ₂₉ w ₃₀ w ₃₁

$$2 \bmod 4 = 2$$

$$4 \bmod 4 = 0$$

Main mly

0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7

Number of blocks
in Main memory

$$M = 8$$

$$= \frac{32 \text{ words}}{4 \text{ words in each block}}$$

$$= 8 //$$

$$\text{block Number} = 1$$

$$1 \text{ mode } 4$$

$$= 1$$

How cpu will get correct word $\rightarrow$ So we require tag bits

Tag field associate with cache memory
it used to identify which block is present in the cache memory

How to put bits in the tag field

tag bit	0, 4
1, 5	
2, 6	
3, 7	

0	1
0	4
1	5
2	6
3	7

If tag bit is 0 = 0
1 = 4

example placed 0th block.

0	0

4 block = 2 bits

0 0 0

0
1
2
3
4

$0 \bmod 4 = 0$

tag
tag 1 0 0 $\leftarrow$ block
main only
8 block so
3 bits required

0 $\rightarrow$ 0 0 0

tag word
block number

Tag set is 000 = 3 bits required $0 \rightarrow 000$

eg $2 \rightarrow 010$
 $\uparrow$ $\uparrow$ $\uparrow$
 tag block = 2

$4 \rightarrow 111$
 $\uparrow$ $\uparrow$ $\uparrow$
 tag 1 3 block

Physical address
 cpu want to access words not block
 Main mty $\rightarrow$ cache mty
 block

cpu access word through physical address

block $\rightarrow$ 4 words

00	}	w ₀	00	$\rightarrow$ w ₄
01		w ₁	01	$\rightarrow$ w ₅
10		w ₂	10	$\rightarrow$ w ₆
11		w ₃	11	$\rightarrow$ w ₇

Physical address divided into 3 positions

Tag	Cache block offset	Word offset
-----	--------------------	-------------

How many bits in our physical address depends on how many words in our Main Memory
 ie 32 words $= 2^5 = 32$

5 bits

5 bits is divided into $\rightarrow$ Tag cache block word off set
 offset

one block = 4 words
 size

so word offset $2^2 = 4 = 2$
 or

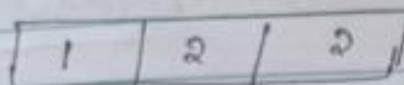
take $\log_2$

How many bits required to word offset

cache block $\rightarrow 2^2 = 4 = 2 // \rightarrow$ which cache block
 Rema

word offset $\rightarrow$ In cache block which word

tag $\rightarrow$ verify whether the particular block present / not



tag =
eg: no by no = 8/4
tag

eg:-

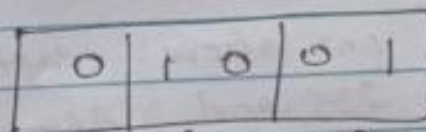
check word 9 is present in cache or not

1st $\rightarrow$ cpu will generate physical address for this word no 9 present

next $\rightarrow$ check whether placed in cache or not

Replace 9 into binary $\rightarrow 1001$

physical address is 5 so add 0.



↑ tag ↑ cache block ↑ word offset

$\rightarrow$ identify which block of cache only the block of main mly can be present

10 $\rightarrow$ 2

0	0, 4	$w_0 w_1 w_2 w_3$
	1, 5	$w_4 w_5 w_6 w_7$
	2, 6	$w_8 w_9 w_{10}$ 0, 1, 2
	3, 7	$w_{11} w_{12} w_{13} w_{14}$

1st identify each block whether placed

↓
by tag $\rightarrow$ tag = 0 yes placed here

block no word offset

$\rightarrow$ which position or the word is placed

Associative Mapping

> Block of ~~the~~ Main Memory can be stored in cache line

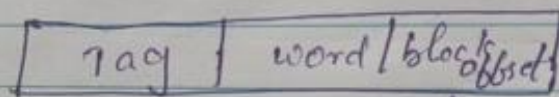
> There is no fixed location for a block

increase flexibility
Reduce conflict Misses

Each cache line $\rightarrow$ tag field $\rightarrow$ store the
my block Number

works

Physical addresses



Identify which
my block is stored



Identify the
word within the
block

The cache controller compare the tag of the cpu address with the tag stored in all cache line (parallel search)

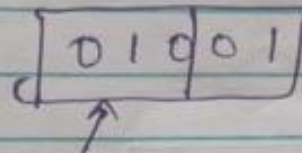
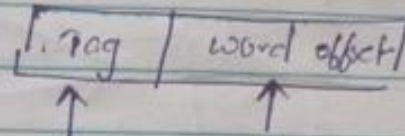
If Match $\rightarrow$ cache hit

no match $\rightarrow$ cache miss

apsara
Date: _____
eg:- Block 5 = 0101 can go any cache line
on look up all tags are compared in parallel

cache 4 line $\frac{M}{My}$ 8 block 0-7
cpu

Search Block 9 $\rightarrow$ 1001



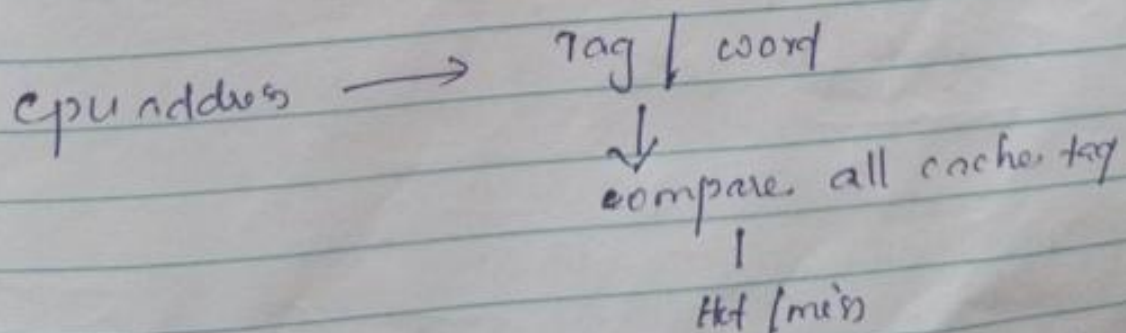
tag $\rightarrow$ Block $\rightarrow$ 1 word

to search block in parallel

Dis adv

Require Associative memory - Hardware expensive

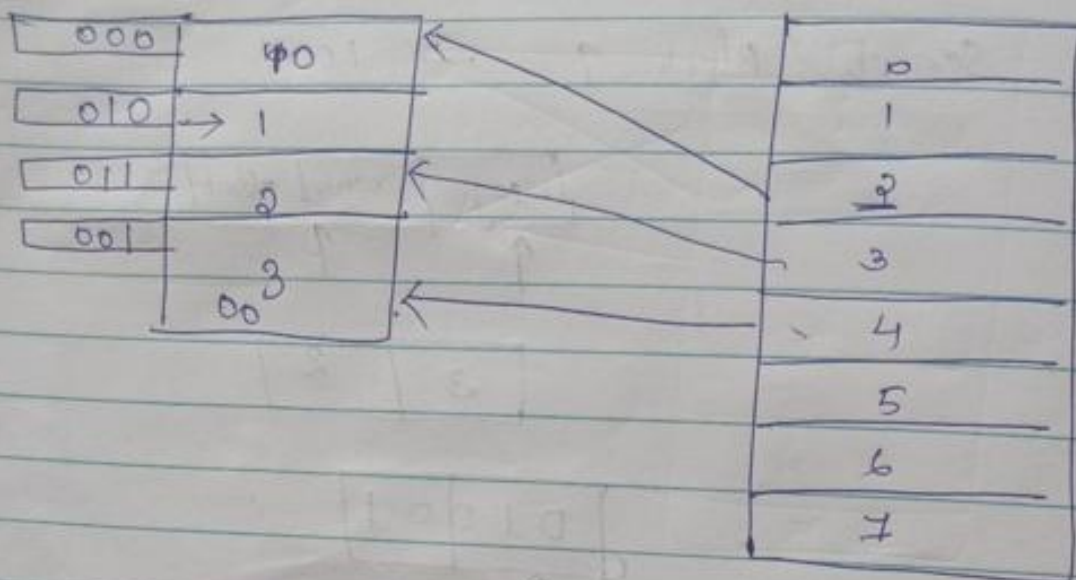
Slower than direct Mapping due to
parallel tag comparisons



Line 0 → tag 0101 mean

000101

↑ decimal 5



00010

00100

1000
2000
3000
4000
5000
6000
7000
8000
9000

Eg:-

9 words. →

01001

↑ ↑

tag word no

So we search parallelly all tag and find the tag and identified the word.

III Set Associative Mapping

- Direct + Associative
- cache divided into sets. Each set have multiple line.
- Memory block can be placed in any line of its mapped set.

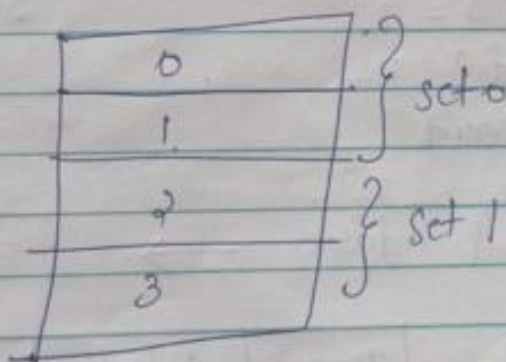
cache = 8 line → 2 way and
 $8/4 = 2$ line

Total Number of Set

$$S = \frac{N}{k} \leftarrow \begin{array}{l} \text{Total Number of Block in cache} \\ \text{Blocks per set} \end{array}$$

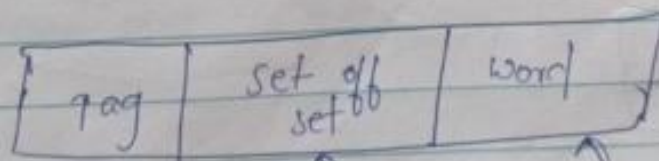
2-way set $S = 48/2 = 24$

no of block
 $32 \rightarrow 8$



working

Physical address 3 part



↑
Identify block

↑
Identify which set in cache
look at

↑
Identify which word inside the block.

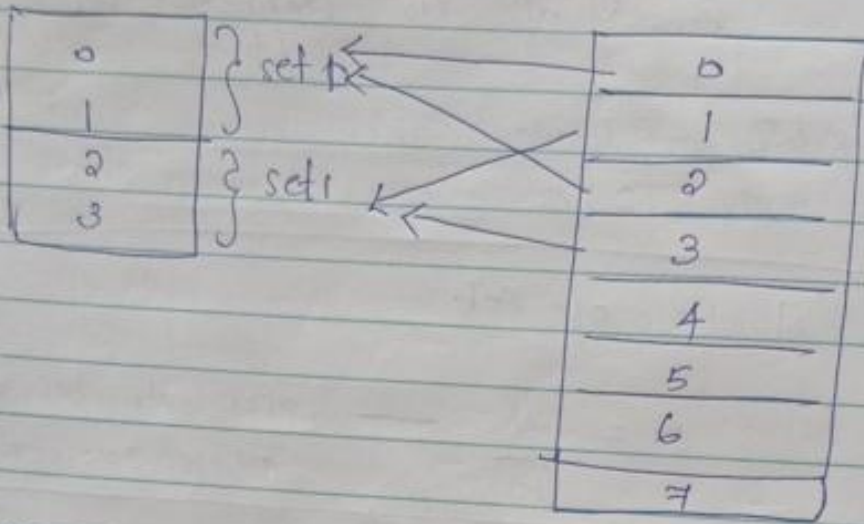
9

011001
 ↑ ↑
 tag Set word

$$k^{th} \text{ block} \rightarrow (k^{th} \bmod S)$$

Number of set

$$\begin{aligned} 0 \bmod 2 &= 0 \\ 2 \bmod 2 &= 0 \\ 1 \bmod 2 &= 1 \end{aligned} \quad \left. \begin{array}{l} \\ \\ \end{array} \right\} \begin{array}{l} \text{Set 0} \\ \text{Set 1} \end{array}$$



$$k^{th} \bmod S$$

$$\begin{aligned} 0 \bmod 2 &= 0 & 3 \bmod 2 &= 1 \\ 1 \bmod 2 &= 1 \\ 2 \bmod 2 &= 0 \end{aligned}$$

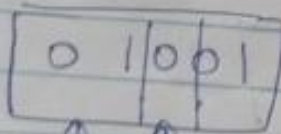
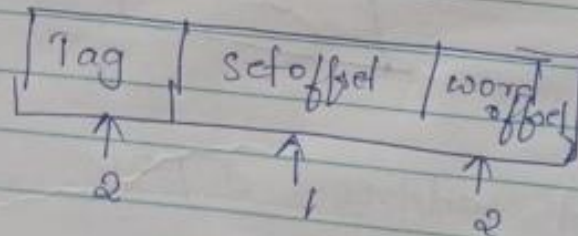
Set offset

$$\begin{aligned} 2^0 &= 1 \\ 2^1 &= 2 \end{aligned}$$

// bit

Eg:- 9 word

Physical address →



tag set word 1

block 1

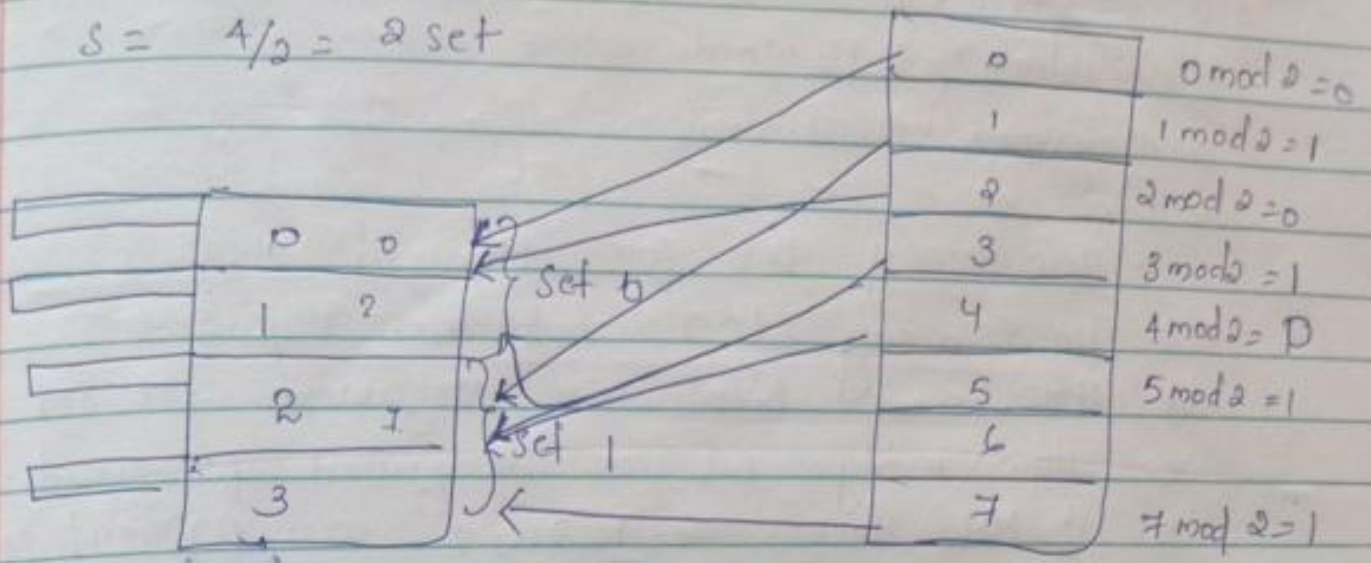
2 way associative

apsara

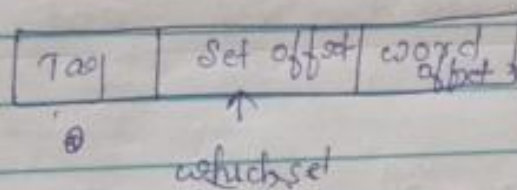
Date: _____

4 blocks

$$S = 4/2 = 2 \text{ set}$$



Physical address



4 words in block $2^2 = 4$ 2//

set offset $\rightarrow$ 1 bit to set $= 2^1 = 2$ 1//

Remaining tag bit = 2//

01 / 010 1
 ↓ ↑
 01 set 0
 0 — 01 ← 1st word.

Ex: MM = 512 word
 CM = 64 words
 Block size = 16 word

$$\frac{512}{16} = 32 \text{ blocks}$$

2 way set associative

1. How many bits for physical address
2. How many tag bits are required
3. How many bits are required for set offset
4. How many bits are required for word offset
5. If MM blocks 16, 9, 26, 7 are present in CM then show tag of each of cache
6. Now check on above cache representation that word 237 and 400 is present or not

1. $512 = 2^9$ — physical address = 9 bits

2. Number of blocks = $512/16 = 32$ blocks

3. Number of blocks in cache = $\frac{64}{16} = 4$ blocks

2 way cache 4 blocks $S = 4/2 = 2$ set

4	1	4
---	---	---

Block 16

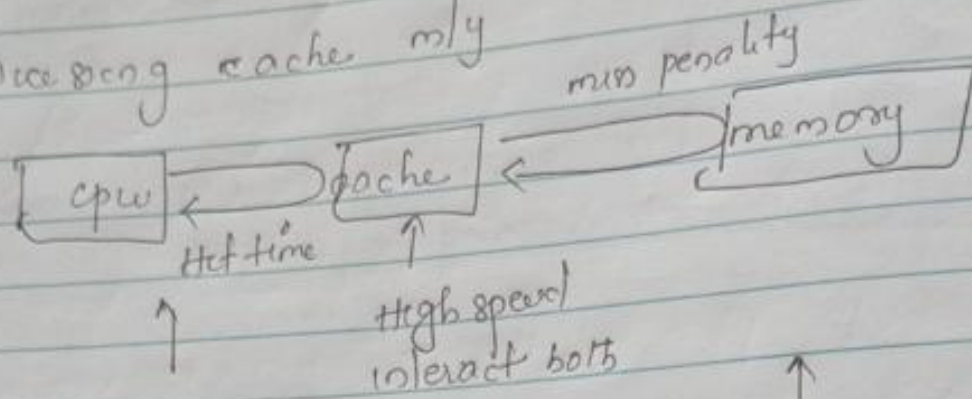
$$24 = 16$$

How to improve performance / optimisation

Date: _____

cache memory optimisation

Accessing cache m/y



Cache Hit Rate → Time Required to perform an access on a cache and return the accessed word is Hit time

(indexing, tag comparison, transfer)

Cache Miss → in certain case Requested address May not be present in the cache
 ↓
 miss rate
 ↓
 avg. time
 memory → called cache memory

cache miss penalty

The extra amount of time required to go to the next level of memory (Main m/y) and bring a block of data and put in the cache memory that is known as Miss penalty.

Average Memory Access time

$$\text{Hit time} + (\text{Miss Rate} \times \text{Miss penalty})$$

3 type cache miss.

How to optimize cache / optimization
Reduce average memory access time technique

3 basic

I 1. Reducing miss penalty rate $\rightarrow$ 3 type caches

II 2. Reducing miss penalty

III 3. Reduce hit rate time.

Reducing Miss Rate $\rightarrow$ of cache mly

I map increasing block size to Reduce miss Rate

• Advantages of larger block

① $\hookrightarrow$ utilize spatial locality :-

Assume a block have n whenever a
 eg 8 words a cpu give an address
 go to search cache if miss then going to
 miss main memory bringing 8 word ^{block} together
 spatial $\rightarrow$ Remain seven block
 locality 8 words together subsequent
 7 word hit

your block size is doubled i.e 16
 Once we encountered a miss 16 block word
 are brought that means if we are going to
 sequential access remaining 15 of them will be
 same hit

② Reduce compulsory misses $\rightarrow$ misses encountered
 very first access of time.

Disadvantages

① increase miss penalty

eg- A block having 4 words
A block having 8 word

Main m/y ~~going~~ connect with a bus

in case 4 words mixing capacity of bus is 4
in one clock cycle, bring 4 words together

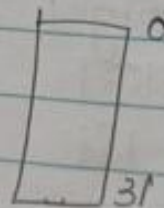
in 8 $\rightarrow$ capacity of channel / bus connect with
cache and Main m/y is not changing
one 4 word is going to 1st cycle &
second 4 word is going to 11th cycle

∞ So increase clock size the bandwidth
connecting cache and main memory unaltered
then more number of m/y cycles are
required in order to carry this this
will effectively increase miss penalty

② More time to fetch a block to the cache (bus width issue)

③ increase conflict misses.

cache m/y $\rightarrow$ 32 block
4 byte each



$\uparrow$ block size

④ More number of blocks mapped into same block location

④ May bring useless data and evict useful cache pollutes.

II Longer cache to reduce miss rate :-

Advantages

↳ Reduces capacity of misses

↳ can accommodate larger memory footprint

Draw back

> Longer H_d rate time → index time ↑

> Higher cost, area and power

III Higher associativity to reduce miss rate :-

> Fully associative cache are the best but high hit rate

> So increase the associativity to an optimal possible level

[good → Direct Map] → 2 way 3 way 4 way
↳ hit rate ↑

increase associativity miss rate will come at same time
hit rate time will increase

Advantages

> Reduce conflict miss

> Reduce miss rate and eviction rate

Draw back
 ↳ increase hit time
 ↳ complex design than direct mapped
 ↳ More time to search in the set

II Miss penalty Reducing Miss penalty

I Multilevel cache

> Multilevel caches to reduce Miss penalty
 > performance gap b/w processor and m/m
 is widening

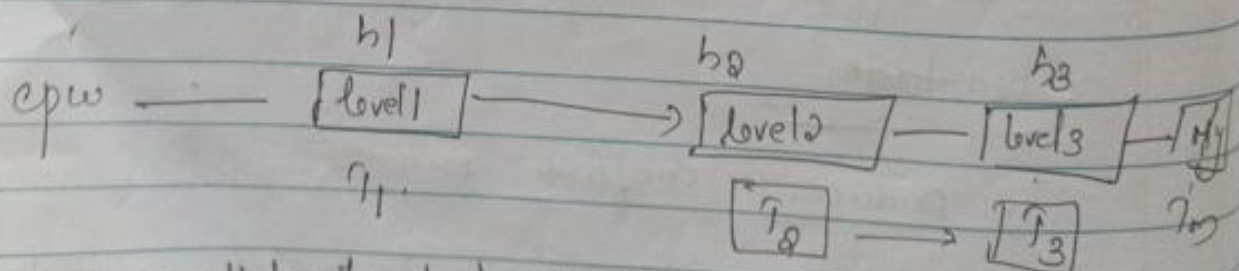
L₁ cache → simple and small

Design

caches should be faster to keep pace with the speed of processors, AND cache should be larger to overcome the widening gap b/w processor and Main Memory

L₁ → Reduce hit

L₂ → Reduce miss penalty (high)



h_x = Hit rate level of x cache

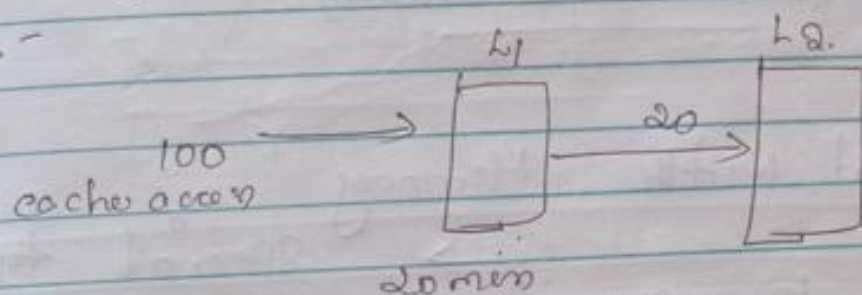
T_x = Access time level x cache / Main m/m

Average access time $\rightarrow$ Hit time L_1 + miss rate $L_1 \times$ miss penalty L_2

Miss $L_1 =$ Hit rate $L_1 +$ miss rate $L_2 \times$ miss rate L_1

- Local miss rate $\rightarrow$ Number of misses in a cache level divided by number of mly access to this level
- global miss rate $\rightarrow$ Number of misses in cache level divided by number of mly access generated by cpu

Eg:-



cpu $\rightarrow$ 100 cache access
out of 20 cache miss in L_1

$$\frac{\text{Number of miss in the cache level}}{\text{number of mly access}} = \frac{20}{100} = 0.2 //$$

There 20 actually going to L_1

Assume out of 20 that reaches to L_2 you are going to have 5 of the missing L_2

$$\begin{aligned} \text{misses in } L_2 &= 5 \\ \text{Access in } L_2 &= 20 \\ \text{Local miss rate } L_2 &= \frac{5}{20} = \frac{1}{4} = 0.25 \end{aligned}$$

global min rate L_3

Min co 5 in L_3 out of 100 given that
cpu has given

$$= \frac{5}{100} = \underline{\underline{0.05}}$$

- Inclusive & exclusive caches

↑ subset of L_2 is taken in L_1	whenever there is L_1 then no is L_3
--	--

Software optimization

High band width Memory

→ special type RAM
designed to provide very high data
transfer rates between the m/c & processor
while consuming less power compared to
conventional m/c e.g. DDR (double data rate)
GDDR (graphics DDR)

Features

3D stacked RAM

consist of multiple memory dies
stacked vertically connected with through
silicon vias (TSV)

side interface

but fast instead of using narrow
bus like GDDR

moderate clock speed, moderate power, band width low.

apsara

Date: _____

Role is AI

especially deep learning & large language model
are memory bandwidth $\rightarrow$ high
 $\rightarrow$ involve massive matrix multiplication
and tensor operations

1. Faster Training and Neural Networks
 $\rightarrow$ Training large model involve
huge data transfer to compute core & only

2. Efficient Inference
 $\rightarrow$ Real time inference e.g. chatbots
Vision system autonomous

3. involve low latency memory access.

4. Scalability in super computing & AI cluster

Basic optimization - Reducing cache miss Rate

cache performance Metrics

Hit Rate - The hit rate is given by the number of cache hit divided by Number of cache access

Miss Rate - The Number of cache misses divided by the number of cache access

$$\text{cache hit} = 1 - \text{Hit Rate} = \frac{\text{no. of misses}}{\text{total access}}$$

Hit time = used to defined the time for hit

miss penalty = defines the extra cost of miss

Average Memory Access Time (AMAT)

$$= \text{Hit Time} + \text{miss rate} \times \text{Miss Penalty}$$

cache misses categories - 3 cs Model / or causes of miss

→ is compulsory → first access to a block is always miss

↓
whatever the situation also called cold start misses
this miss is going to happen misses in infinity size cache

counting the number of misses in hypothetical cache of infinite size

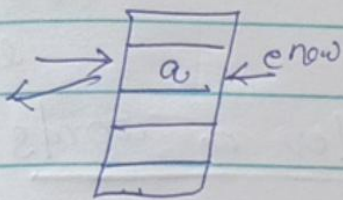
apsara

Date:

2. capacity

— cache cannot contain all blocks needed, capacity misses occur due to block being discarded and later retrieved.

using Direct Mapping



— misses in fully associative cache with optimal replacement

3. Conflict Misses —

Misses occurring because several blocks map to same set

Placement in set

occur set associative & direct Mapping

Associative Mapping / direct

Also called collision Misses



more than one block map

to same set / location

cache block collide the same set

collision misses / interference

So we can improve cache performance (improve cache perform)

coherence

↳ multiple processors
↳ multiple coherence.

$$AMAT = HitTime + missRate \times missPenalty$$

①

• Reduce Hit Time

→ Small & simple cache
Avoid Address translation cohen

devices with direct memory access (DMA) ability may be accessing information in other memory modules.

The second and more effective way to address the modules is shown in Figure 5.24b. It is called *memory interleaving*. The low-order k bits of the memory address select a module, and the high-order m bits name a location within that module. In this way, consecutive addresses are located in successive modules. Thus, any component of the system that generates requests for access to consecutive memory locations can keep several modules busy at any one time. This results in both faster access to a block of data and higher average utilization of the memory system as a whole. To implement the interleaved structure, there must be 2^k modules; otherwise, there will be gaps of nonexistent locations in the memory address space.

The effect of interleaving is substantial. Consider the time needed to transfer a block of data from the main memory to the cache when a read miss occurs. Suppose that a cache with 8-word blocks is used, similar to our examples in Section 5.5. On a read miss, the block that contains the desired word must be copied from the memory into the cache. Assume that the hardware has the following properties. (It takes one clock cycle to send an address to the main memory. The memory is built with DRAM chips that allow the first word to be accessed in 8 cycles, but subsequent words of the block are accessed in 4 clock cycles per word. (Recall from Section 5.2.3 that, when consecutive locations in a DRAM are read from a given row of cells, the row address is decoded only once. Addresses of consecutive columns of the array are then applied to access the desired words, which takes only half the time per access.) Also, one clock cycle is needed to send one word to the cache.

If a single memory module is used, then the time needed to load the desired block into the cache is

$$1 + 8 + (7 \times 4) + 1 = 38 \text{ cycles}$$

Suppose now that the memory is constructed as four interleaved modules, using the scheme in Figure 5.24b. When the starting address of the block arrives at the memory, all four modules begin accessing the required data, using the high-order bits of the address. After 8 clock cycles, each module has one word of data in its DBR. These words are transferred to the cache, one word at a time, during the next 4 clock cycles. During this time, the next word in each module is accessed. Then it takes another 4 cycles to transfer these words to the cache. Therefore, the total time needed to load the block from the interleaved memory is

$$1 + 8 + 4 + 4 = 17 \text{ cycles}$$

Thus, interleaving reduces the block transfer time by more than a factor of 2.

5.6.2 Hit Rate and Miss Penalty

An excellent indicator of the effectiveness of a particular implementation of the memory hierarchy is the success rate in accessing information at various levels of the hierarchy. Recall that a successful access to data in a cache is called a hit. The number of hits stated as a fraction of all attempted accesses is called the *hit rate*, and the *miss rate* is the number of misses stated as a fraction of attempted accesses.

Ideally, the entire memory hierarchy would appear to the CPU as a single memory unit that has the access time of a cache on the CPU chip and the size of a magnetic disk. How close we get to this ideal depends largely on the hit rate at different levels of the hierarchy. High hit rates, well over 0.9, are essential for high-performance computers.

Performance is adversely affected by the actions that must be taken after a miss. The extra time needed to bring the desired information into the cache is called the *miss penalty*. This penalty is ultimately reflected in the time that the CPU is stalled because the required instructions or data are not available for execution. In general, the miss penalty is the time needed to bring a block of data from a slower unit in the memory hierarchy to a faster unit. The miss penalty is reduced if efficient mechanisms for transferring data between the various units of the hierarchy are implemented. The previous section shows how an interleaved memory can reduce the miss penalty substantially.

Consider now the impact of the cache on the overall performance of the computer. Let h be the hit rate, M the miss penalty, that is, the time to access information in the main memory, and C the time to access information in the cache. The average access time experienced by the CPU is

$$t_{ave} = hC + (1 - h)M$$

We use the same parameters as in the example in Section 5.6.1. If the computer has no cache, then, using a fast processor and a typical DRAM main memory, it takes 10 clock cycles for each memory read access. Suppose the computer has a cache that holds 8-word blocks and an interleaved main memory. Then, as we show in Section 5.6.1, 17 cycles are needed to load a block into the cache. Assume that 30 percent of the instructions in a typical program perform a read or a write operation, and assume that the hit rates in the cache are 0.95 for instructions and 0.9 for data. Let us further assume that the miss penalty is the same for both read and write accesses. Then, a rough estimate of the improvement in performance that results from using the cache can be obtained as follows:

$$\frac{\text{Time without cache}}{\text{Time with cache}} = \frac{130 \times 10}{100(0.95 \times 1 + 0.05 \times 17) + 30(0.9 \times 1 + 0.1 \times 17)} = 5.04$$

This result suggests that the computer with the cache performs five times better.

It is also interesting to consider how effective this cache is compared to an ideal cache that has a hit rate of 100 percent (in which case, all memory references take one cycle). Our rough estimate of relative performance for these caches is

$$\frac{100(0.95 \times 1 + 0.05 \times 17) + 30(0.9 \times 1 + 0.1 \times 17)}{130} = 1.98$$

This means that the actual cache provides an environment in which the CPU effectively works with a large DRAM-based main memory that is only half as fast as the circuits in the cache.

In the preceding example, we distinguish between instructions and data as far as the hit rate is concerned. Although hit rates above 0.9 are achievable for both, the hit rate for instructions is usually higher than that for data. The hit rates depend on the design of the cache and on the instruction and data access patterns of the programs being executed.

How can the hit rate be improved? An obvious possibility is to make the cache larger, but this entails increased cost. Another possibility is to increase the block size while keeping the total cache size constant, to take advantage of spatial locality. If all items in a larger block are needed in a computation, then it is better to load these items into the cache as a consequence of a single miss, rather than loading several smaller blocks as a result of several misses. The efficiency of parallel access to blocks in an interleaved memory is the basic reason for this advantage. Larger blocks are effective up to a certain size, but eventually any further improvement in the hit rate tends to be offset by the fact that, in a larger block, some items may not be referenced before the block is ejected (replaced). The miss penalty increases as the block size increases. Since the performance of a computer is affected positively by increased hit rate and negatively by increased miss penalty, the block sizes that are neither very small nor very large give the best results. In practice, block sizes in the range of 16 to 128 bytes have been the most popular choices.

Finally, we note that the miss penalty can be reduced if the load-through approach is used when loading new blocks into the cache. Then, instead of waiting for the completion of the block transfer, the CPU can continue as soon as the required word is loaded in the cache. This approach is used in the PowerPC processor.

5.6.3 Caches on the CPU Chip

When information is transferred between different chips, considerable delays are introduced in driver and receiver gates on the chips. Thus, from the speed point of view, the optimal place for a cache is on the CPU chip. Unfortunately, space on the CPU chip is needed for many other functions; this limits the size of the cache that can be accommodated.